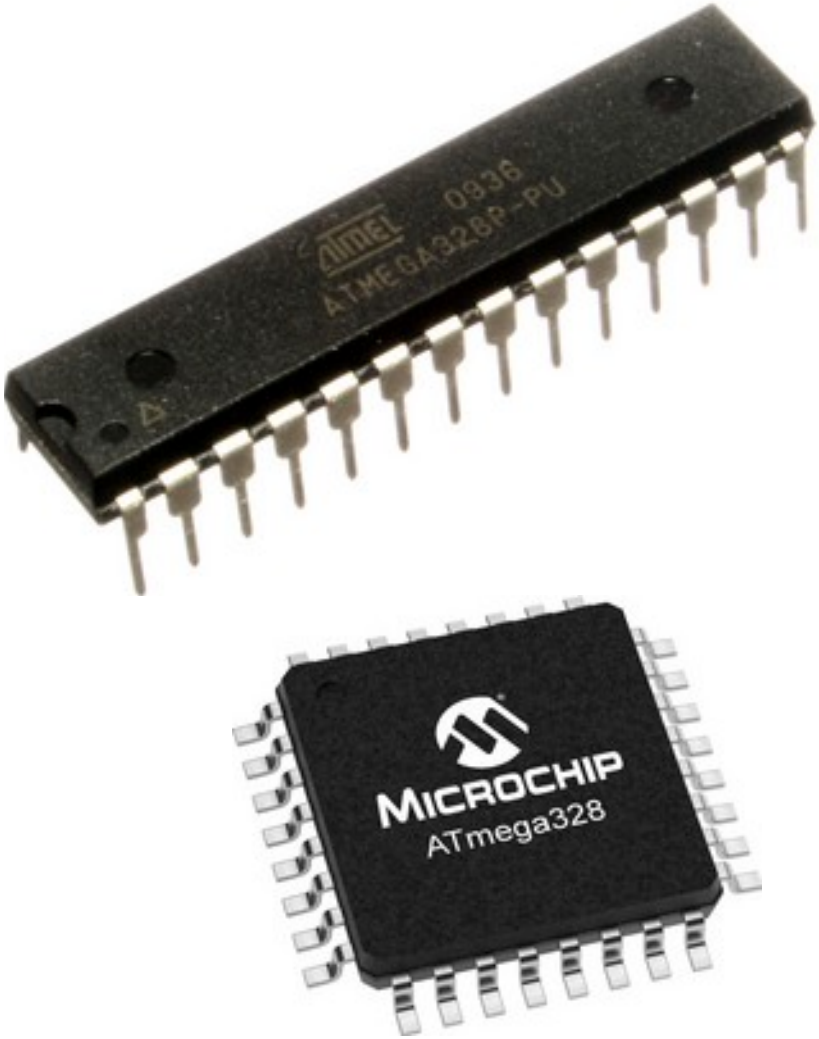
Introdução ao microprocessamento

Processadores

Intel Core i9



Microchip ATmega328

Características	Microprocessadores	Microcontroladores
Periféricos	Necessita de periféricos externos.	Memória e periféricos integrados no chip.
Barramento	Largura elevada do barramento de endereços.	Barramento de endereçamento estreito só permitindo o gerenciamento de pequena quantidade de memória.
Gerenciamento de memória	Unidade de gerenciamento de memória integrada, possibilitando vários formatos de dados.	Tratamento de poucos formatos de dados (tipicamente 8, 16 e 32 bits).
Processamento	Coprocessador matemático e unidade lógica aritmética com suporte a operações numéricas complexas, tais como multiplicação em ponto flutuante.	Ausência de coprocessador e com unidade lógica aritmética limitada.
Endereçamento	Sofisticados modos de endereçamento para eficiente suporte às funções de linguagem em alto nível.	Limitados modos de endereçamento (tratamento ineficiente de funções de linguagem de alto nível).
Custo	Alto custo.	Baixo custo.
Consumo	Alto consumo de energia.	Baixo consumo de energia.
Ocupação no silício	Ocupação de grande área de silício.	Pequena área de silício.

Principal diferença

- Funcionalidades e aplicações

Microprocessadores

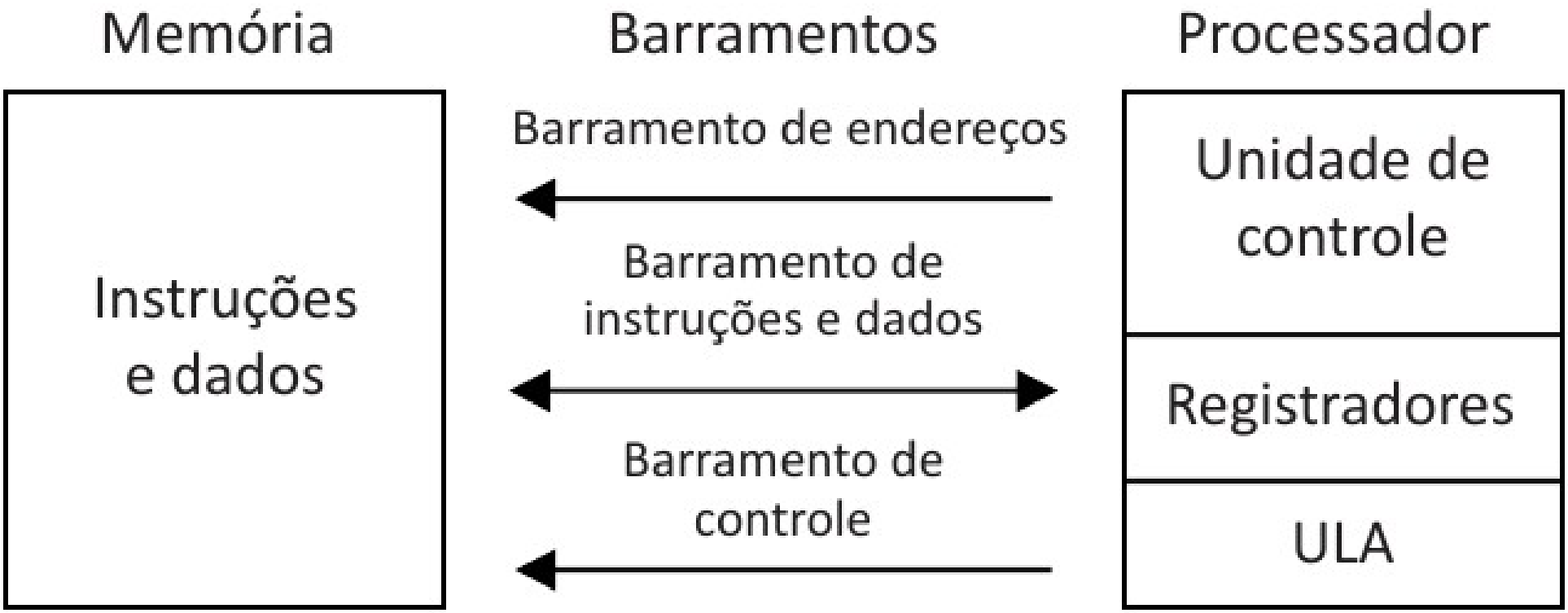
- Grande capacidade de processamento
- Necessita memória externa e periféricos

Microcontroladores

- Utilização mais específica como sistemas embarcados
- Relativa baixa capacidade de processamento
- Tenta minimizar componentes externos
- Memória e periféricos integrados

Arquitetura Von Neumann

Um único sistema de memória



Unidade de controle: busca de instruções e dados na memória, e execução

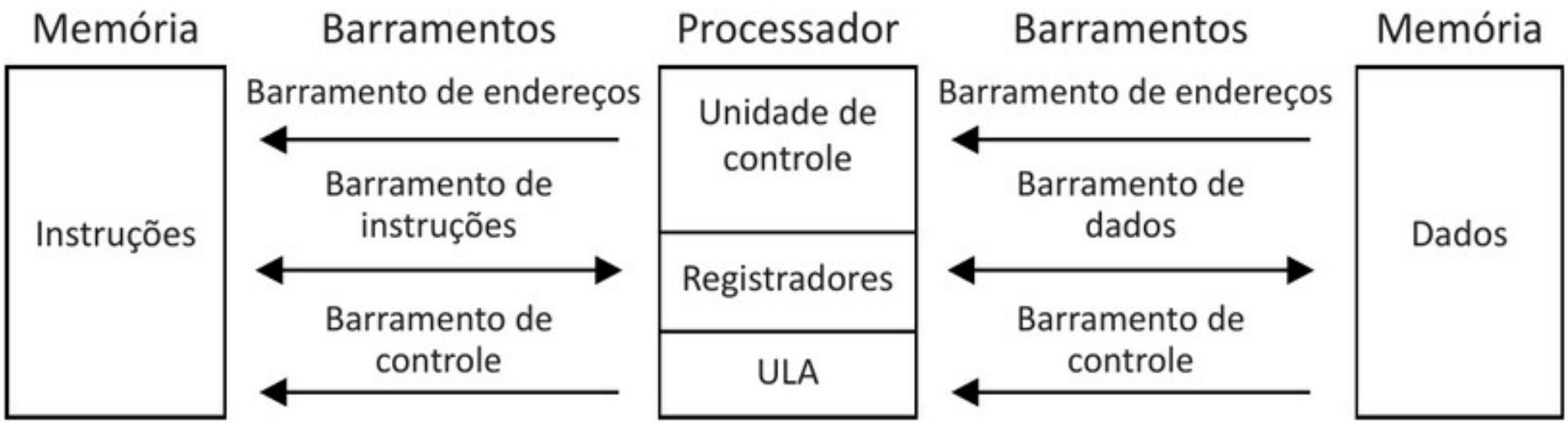
Arquitetura Harvard

2 unidades de memória independentes

uma para instruções, e outro para dados

2 conjuntos de barramentos

dados: 8 bits; instruções: 12, 14, e 16 bits



Harvard x Von Neumann

Harvard

- Acesso simultâneo a dados e instruções pelo processador
- Tipos e tamanhos de memória diferentes
- Mais usado em microcontroladores

Von Neumann

- **Vantagem**: como compartilha barramento, os programas podem ser alterados como se fossem dados
  - Ex.: um compilador para uma linguagem
- **Desvantagem**: a velocidade é afetada devido à ausência de acesso simultâneo a dados e instruções
- Mais usado em processadores para desktops/laptops

Firmware

- Software armazenado no dispositivo e que gerencia funções básicas do hardware

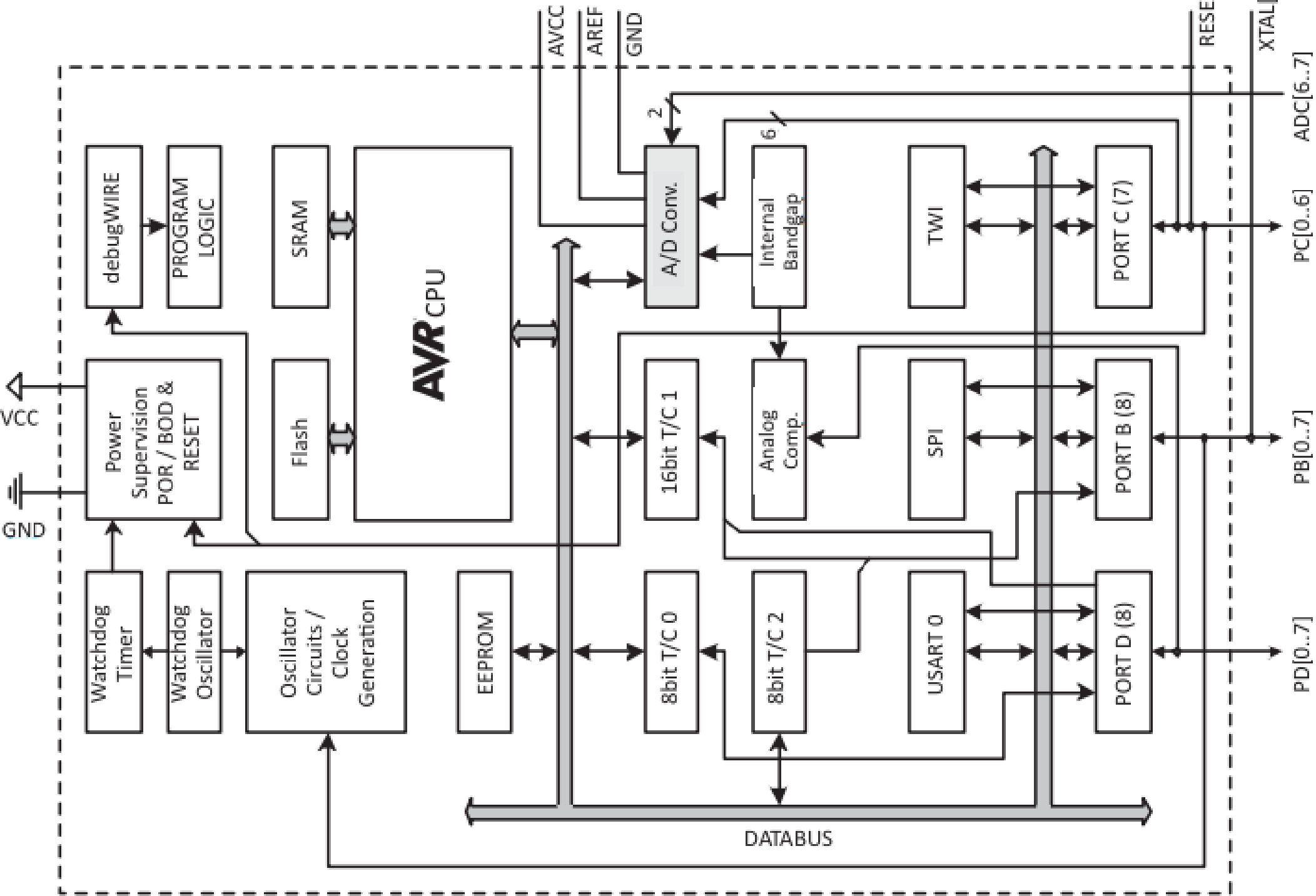
DSP (Digital Signal Processor)

- Microprocessador especializado em processamento digital de sinais

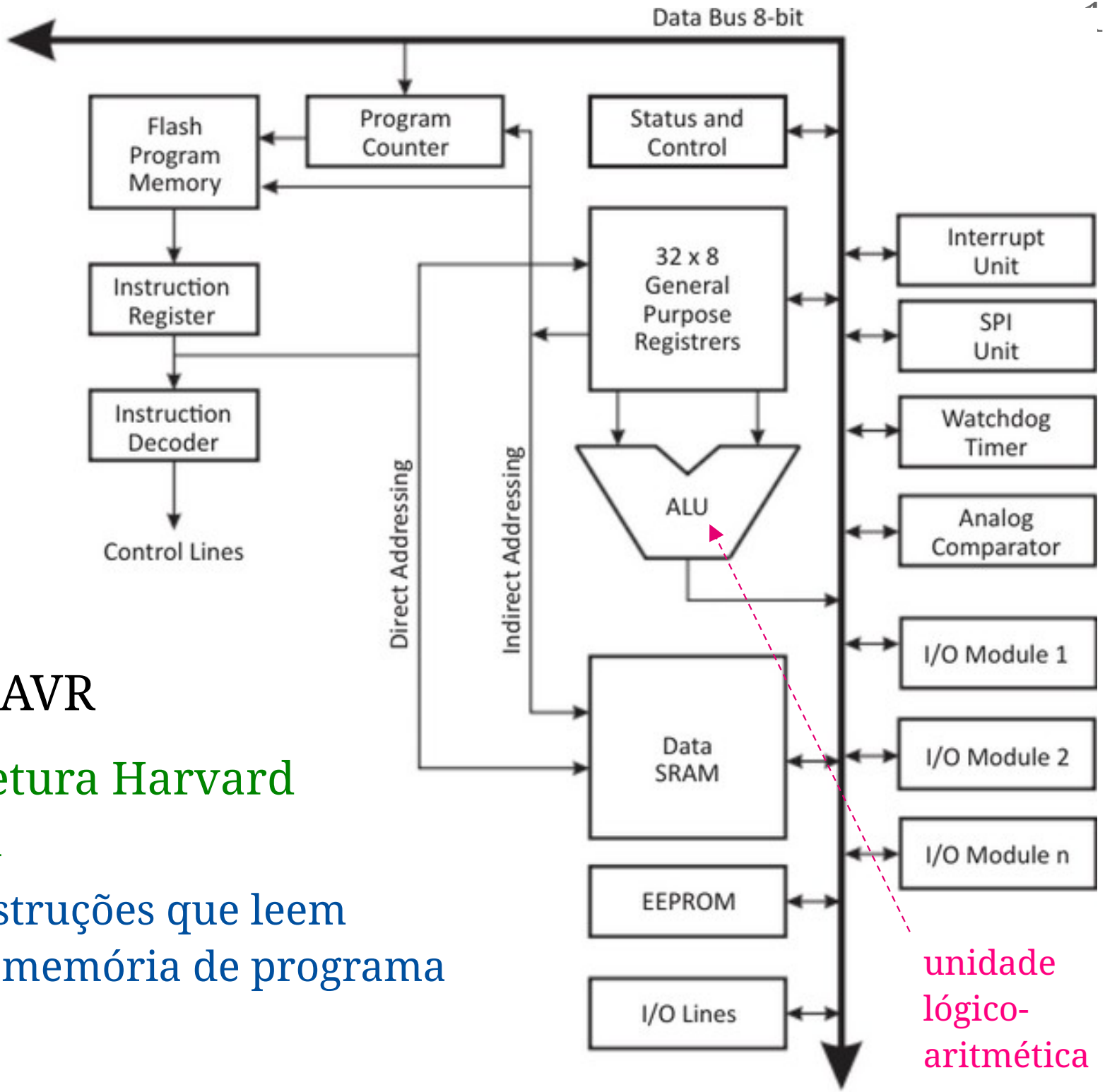
CPU (Central Processing Unit), ou núcleo, ou core

- Menor unidade de execução do processador mais importante
  - Capaz de executar um programa
- Processadores modernos possuem várias CPUs

Microchip ATmega328



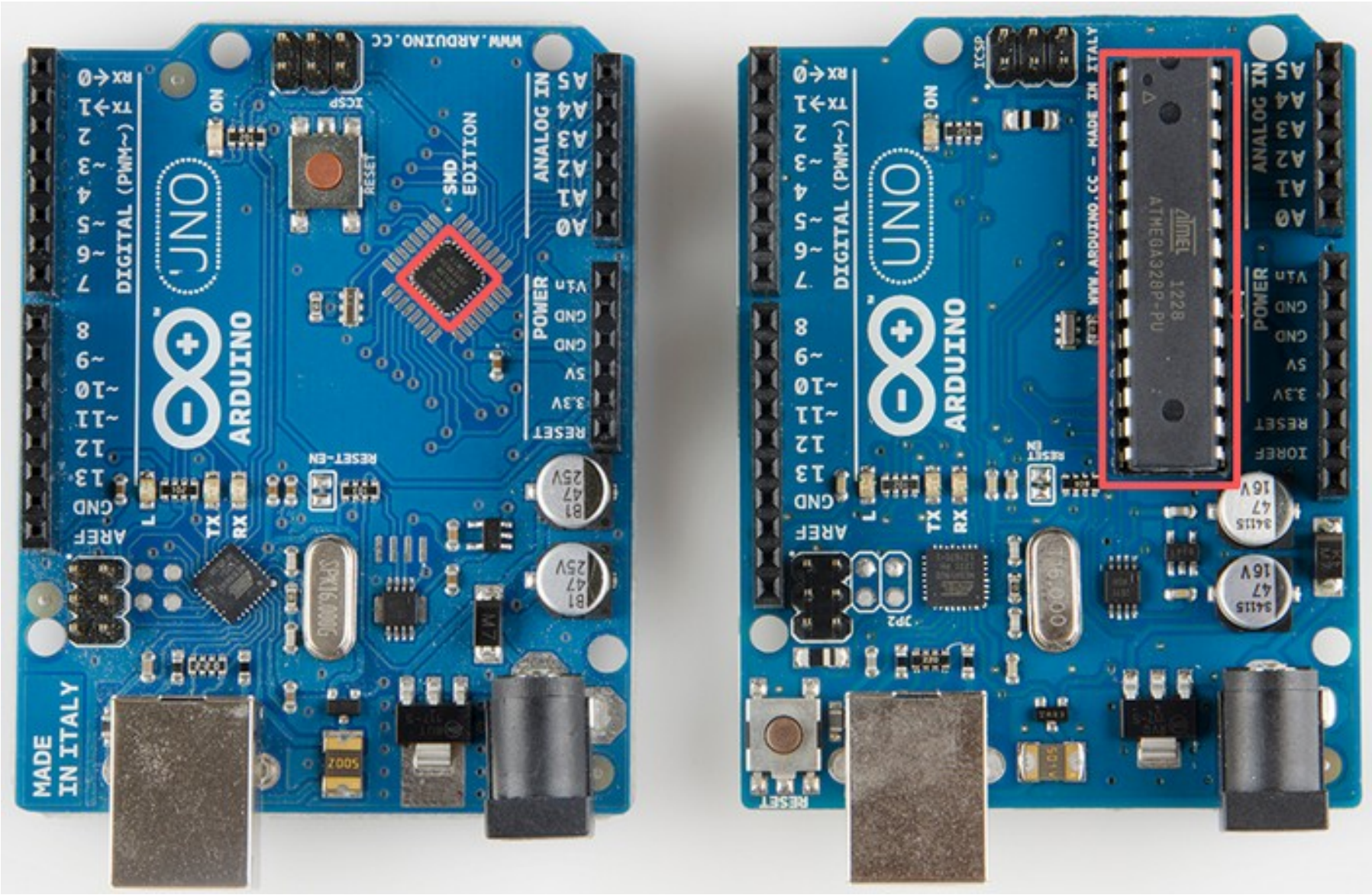
AVR CPU



Arquitetura AVR

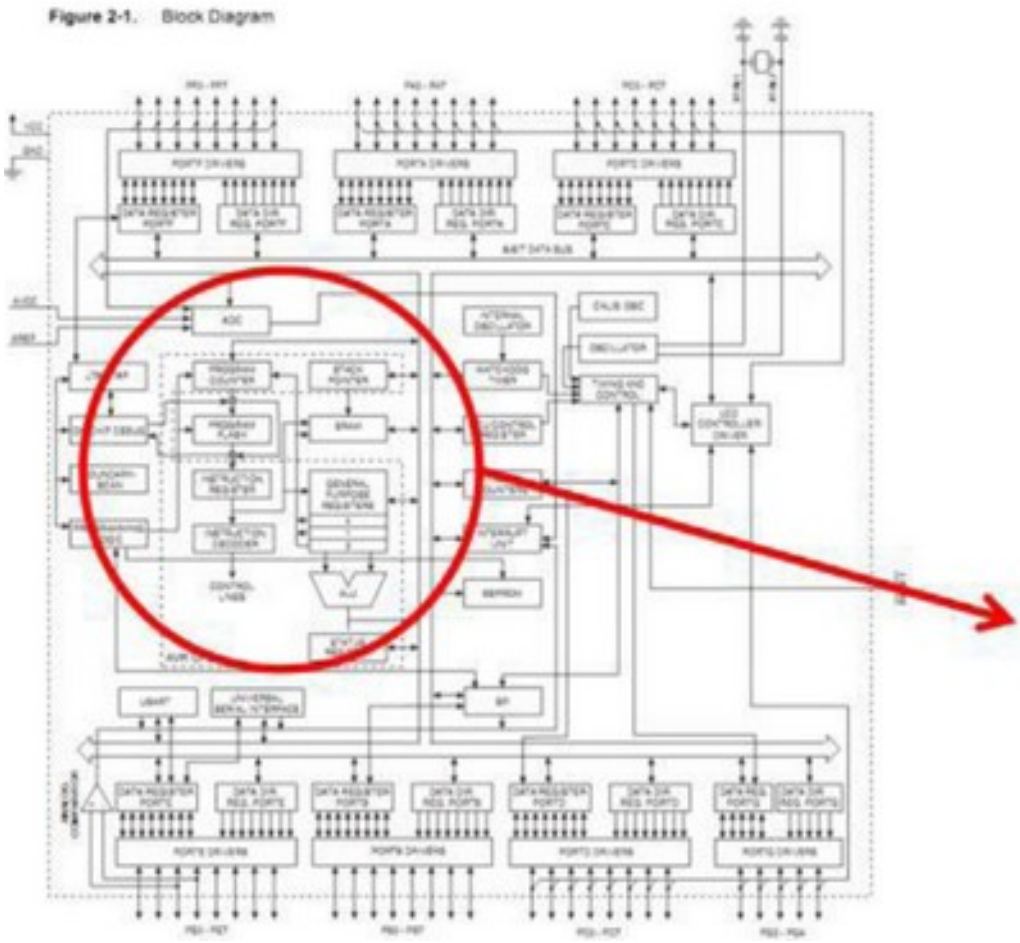
- Usa arquitetura Harvard modificada
  - Possui instruções que leem dados da memória de programa

ATmega328

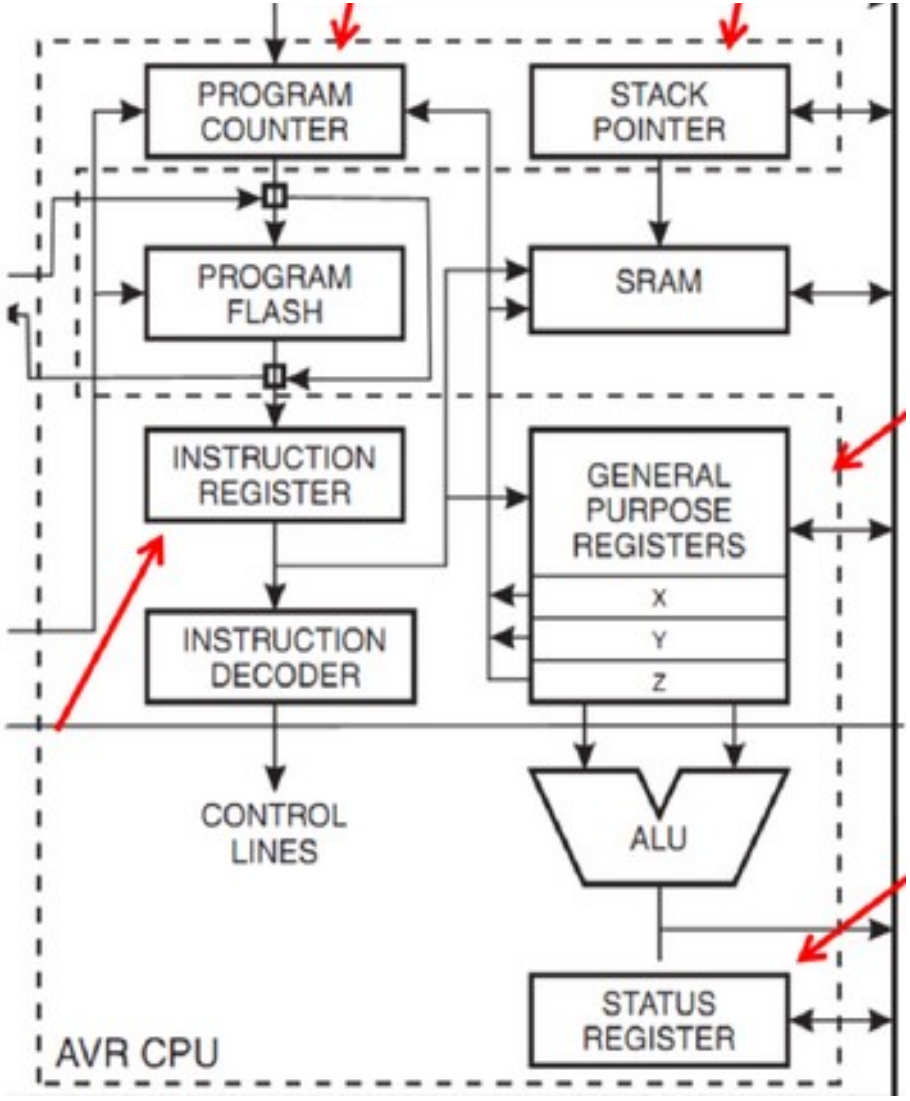

ATMEGA328P-PU
ARDUINO
UNO
DIGITAL (PWM~)
ANALOG IN
POWER
ICSP
RESET
ON
L
TX
RX
MADE IN ITALY
WWW.ARDUINO.CC

Registradores da CPU

Diagrama de blocos



Detalhe



https://eclipse.umbc.edu/robucci/cmpe311/Lectures/L04-AVR_CPU_Registers/

O programa que é executado diretamente pela CPU é armazenado na memória em bytes na forma de código de máquina (números)

Estes códigos possuem significado, que são as instruções de linguagem de máquina

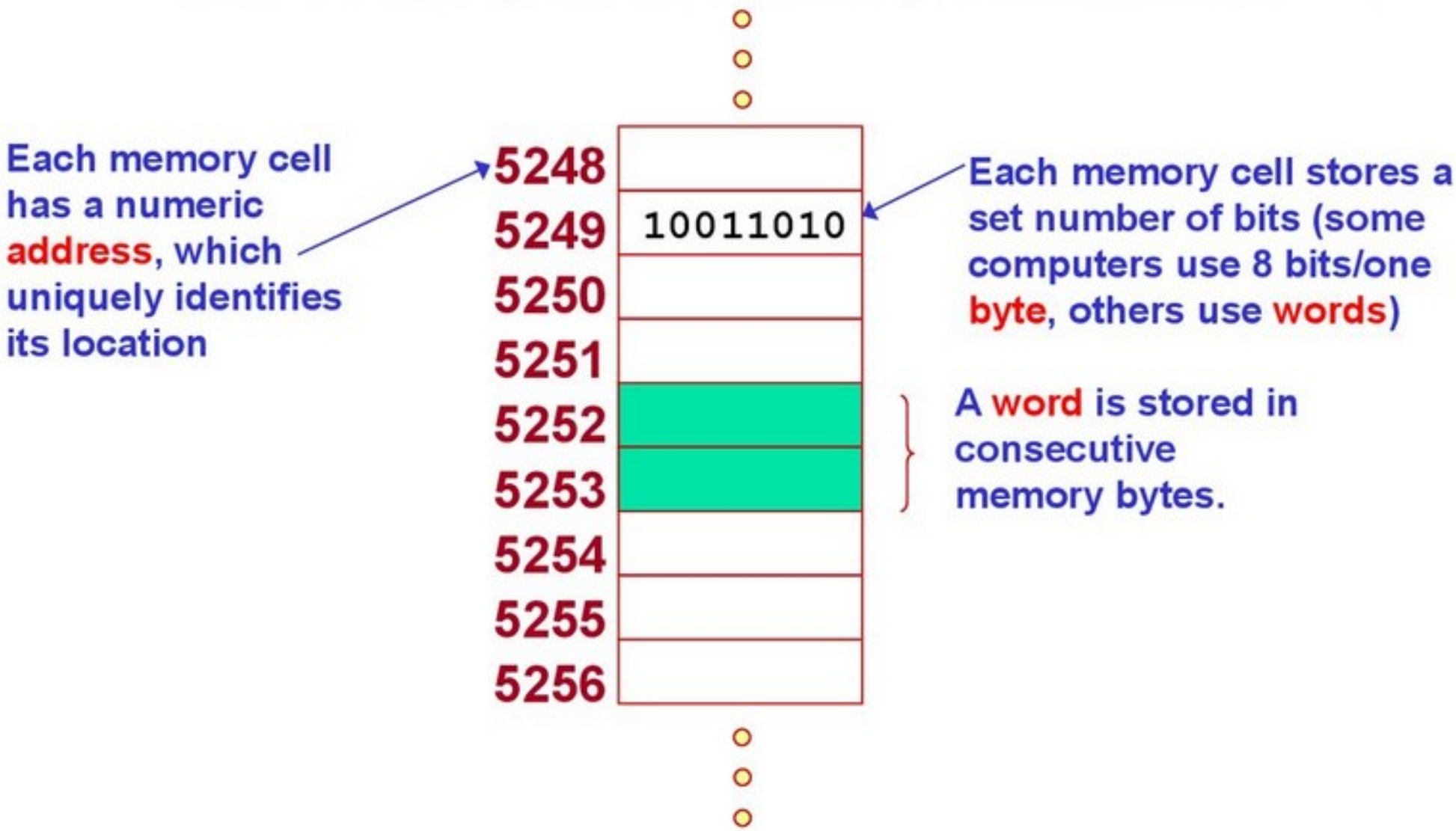
São armazenados em posições sucessivas de memória

A próxima instrução a ser executada pela CPU está uma posição à frente da instrução atual na memória

Código de máquina x Assembly

Execução das instruções

```
8048374:    55                      push   %ebp
8048375:    89 e5                   mov    %esp,%ebp
8048377:    83 ec 08                sub    $0x8,%esp
804837a:    83 e4 f0                and    $0xfffffff0,%esp
804837d:    b8 00 00 00 00          mov    $0x0,%eax
8048382:    29 c4                   sub    %eax,%esp
8048384:    c7 45 fc 00 00 00 00    movl   $0x0,0xfffffffc(%ebp)
804838b:    83 7d fc 09             cmpl   $0x9,0xfffffffc(%ebp)
804838f:    7e 02                   jle    8048393 <main+0x1f>
8048391:    eb 13                   jmp    80483a6 <main+0x32>
8048393:    c7 04 24 84 84 04 08    movl   $0x8048484,(%esp)
804839a:    e8 01 ff ff ff          call   80482a0 <printf@plt>
804839f:    8d 45 fc                lea    0xfffffffc(%ebp),%eax
80483a2:    ff 00                   incl   (%eax)
80483a4:    eb e5                   jmp    804838b <main+0x17>
80483a6:    c9                      leave
80483a7:    c3                      ret
80483a8:    90                      nop
80483a9:    90                      nop
80483aa:    90                      nop
```


Each memory cell has a numeric address, which uniquely identifies its location
5248
5249
10011010
5250
5251
5252
5253
5254
5255
5256
Each memory cell stores a set number of bits (some computers use 8 bits/one byte, others use words)
A word is stored in consecutive memory bytes.

Ride7 - main.c

File Edit View Project Debug Scripts Options Window Help

Debug

- testR7
  - Data Dump
    - Disassembly View
    - Code View
    - Registers View
    - Memory View
    - Memory (16-bit) View
    - Memory (32-bit) View
  - Main Registers
  - Peripherals
    - IOPORT0
    - IOPORT1
    - IOPORT2
    - UART0-buffer
    - UART1-buffer
    - UART2-buffer
    - UART3-buffer
    - Power/Clock/Reset
    - RTC
    - Watchdog
    - Timer0
    - Timer1
    - Timer2
    - Timer3

Project | Globals | Debug

main.c

CodeC...

```
int main()
    {
    //delay for debug
```

Disassembly View [testR7]

search symbol

Hex

Addr...	Symbol	C...	Mnemonic		C...	Last Acti...
4...		20	db 20h ; ' '		0	--
	{					
4...	main	0...	STR	LR, ...	0	--
4...		0...	SUB	SP, ...	0	--
	while(j--);					
4...		3...	LDR	R3, ...	0	--
4...		0...	LDR	R2, ...	0	--
4...		0...	SUB	R2, ...	0	--

Disassembly View [testR7] | Disassembly View [testR7]

Memory View:210 [testR7]

search symbol

```
00000000: 18 F0 9F E5 18 F0 9F E5 . .
00000008: 18 F0 9F E5 18 F0 9F E5 . .
00000010: 18 F0 9F E5 00 00 A0 E1 . .
00000018: 18 F0 9F E5 18 F0 9F E5 . .
00000020: 20 04 00 40 C0 00 00 40   .
00000028: DC 00 00 40 38 01 00 40 . .
00000030: 58 01 00 40 00 00 00 00 X .
00000038: F8 00 00 40 78 01 00 40 . .
00000040: 98 01 00 40 B0 01 00 40 . .
00000048: C8 01 00 40 E0 01 00 40 . .
00000050: F8 01 00 40 10 02 00 40 . .
00000058: 28 02 00 40 40 02 00 40 ( .
00000060: 58 02 00 40 70 02 00 40 X .
00000068: 88 02 00 40 A0 02 00 40 . .
00000070: B8 02 00 40 D0 02 00 40 . .
00000078: E8 02 00 40 00 03 00 40 . .
00000080: 18 03 00 40 30 03 00 40 . .
00000088: 48 03 00 40 60 03 00 40 H .
00000090: 78 03 00 40 90 03 00 40 x .
00000098: 00 00 00 00 00 00 00 00 . .
000000A0: 00 00 00 00 A8 03 00 40 . .
000000A8: C0 03 00 40 00 00 00 00 . .
000000B0: 00 00 00 00 D8 03 00 40 . .
000000B8: F0 03 00 40 08 04 00 40 . .
000000C0: FF 5F 2D E9 00 10 4F E1 . _
000000C8: 02 00 2D E9 CB FF FF EB . .
000000D0: 02 00 BD E8 01 F0 6F E1 . .
000000D8: FF 9F FD E8 FF 5F 2D E9 . .
000000E0: 00 10 4F E1 02 00 2D E9 . .
000000E8: C4 FF FF EB 02 00 BD E8 . .
000000F0: 01 F0 6F E1 FF 9F FD E8 . .
000000F8: 04 E0 4E E2 FF 5F 2D E9 . .
00000100: 00 10 4F E1 02 00 2D E9 . .
```

Memory View:210 [testR7] | Memory View:210 [testR7]

Debug Output

PC	Time	Description
-	14:44:32:281	Starting debug process (Clock frequency: 8.000000 MHz)
40000000	14:44:32:625	Loading C:\Program Files\Raisonance\Ride\examples\ARM\test\testR7.elf
40000000	14:44:32:640	Loading
40000000	0s.000ms.000	Run
40001A10	0s.000ms.650	Stop at Breakpoint

Build Log | Debug Output

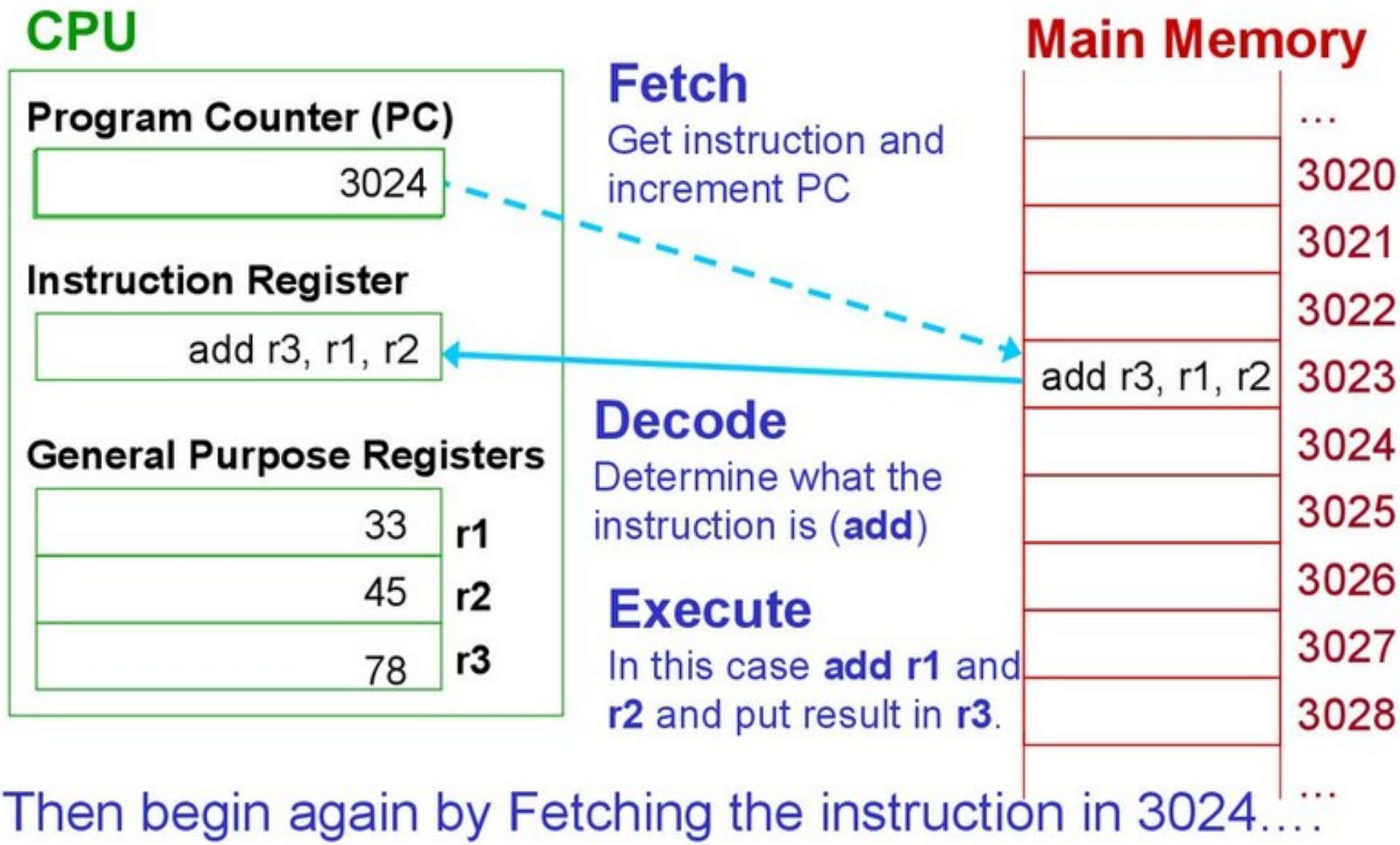
Ready 0s.000ms.650 334:1 CAP NUM INS

A CPU possui ciclos com etapas de busca, decodificação, e execução da instrução

A CPU possui um registrador que é o contador de programa (PC), que armazena o endereço de memória da próxima instrução

Caso o sistema seja reiniciado, o PC é carregado com o endereço da primeira instrução a ser executada

- Esse endereço depende do processador e configurações



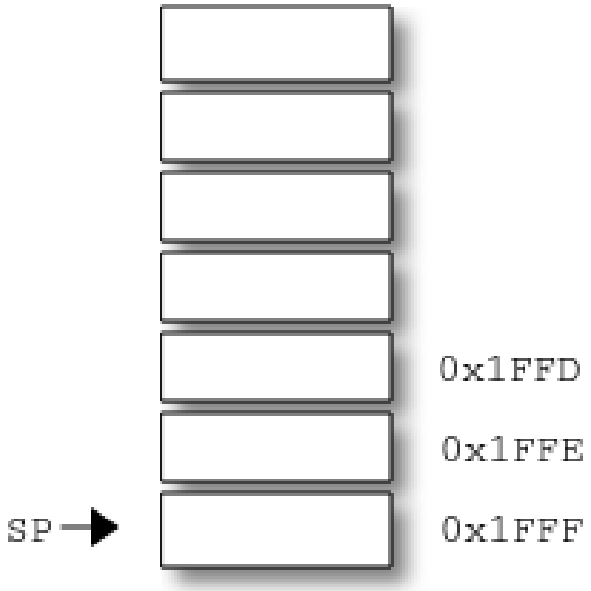
Then begin again by Fetching the instruction in 3024…

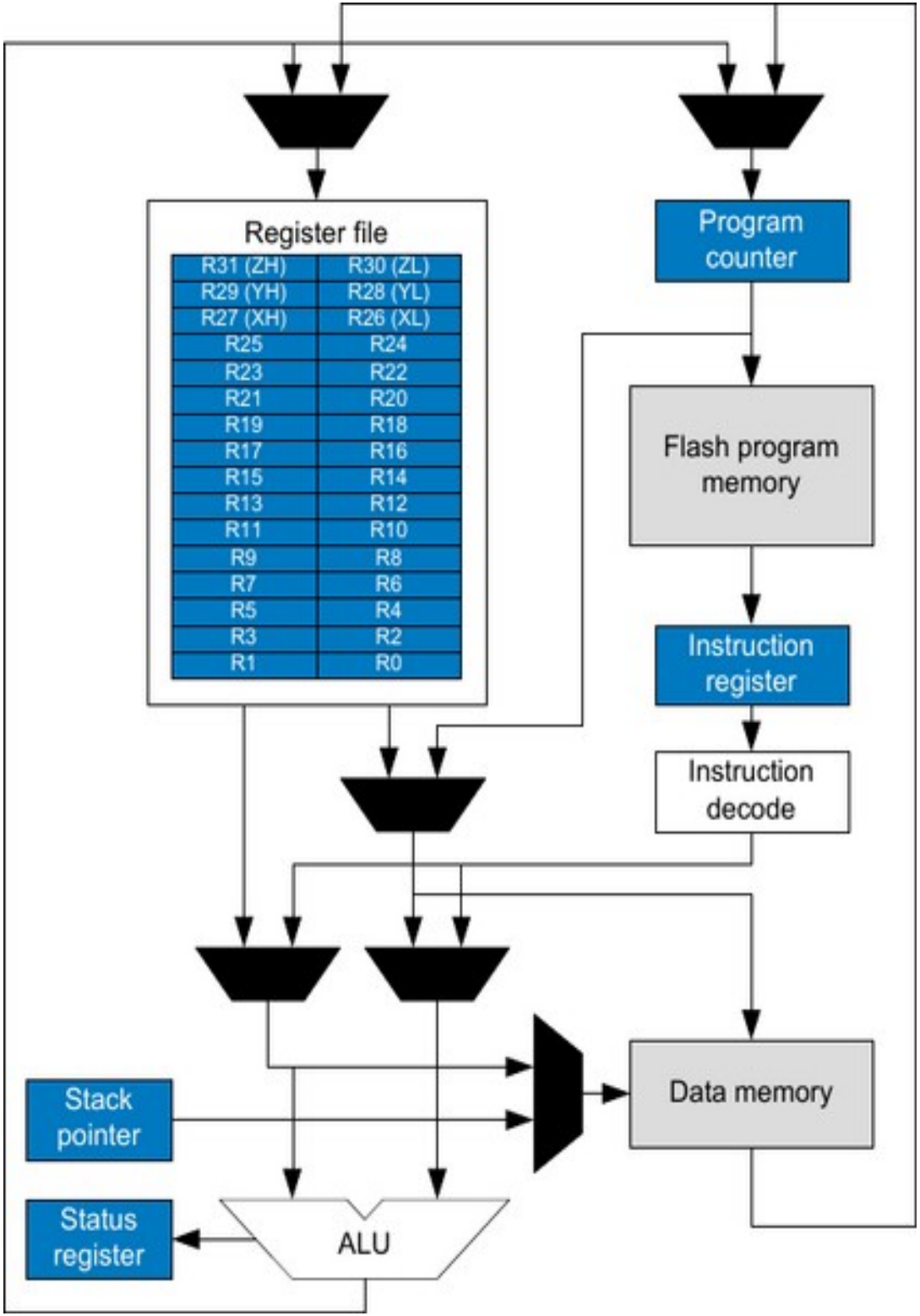
Stack Pointer (SP)

Controlado por hardware

Usado para apontar para o endereço de memória do último elemento adicionado a uma estrutura de dados ordenada

Usado por exemplo para armazenar o endereço de retorno de uma sub-rotina, ou armazenar dados





Instruction Register (IR)

- Contém o conteúdo da memória (instrução) apontada pelo PC

ALU

- Operações lógicas e Aritméticas

Registrador de Status

Status Register

Bit	7	6	5	4	3	2	1	0
Flag	I	T	H	S	V	N	Z	C

The representation of the 8 bits in the status register are:

- Bit 0: Carry Flag
- Bit 1: Zero Flag
- Bit 2: Negative Flag
- Bit 3: Two's Complement Overflow Flag
- Bit 4: Sign Bit
- Bit 5: Half Carry Flag
- Bit 6: Bit Copy Storage
- Bit 7: Global Interrupt Enable

http://www.rjhcoding.com/avr-asm-sreg.php

Registradores de uso geral

7 0	Addr.
R0	0x00
R1	0x01
R2	0x02
...	
R13	0x0D
R14	0x0E
R15	0x0F
R16	0x10
R17	0x11
...	
R26	0x1A
R27	0x1B
R28	0x1C
R29	0x1D
R30	0x1E
R31	0x1F

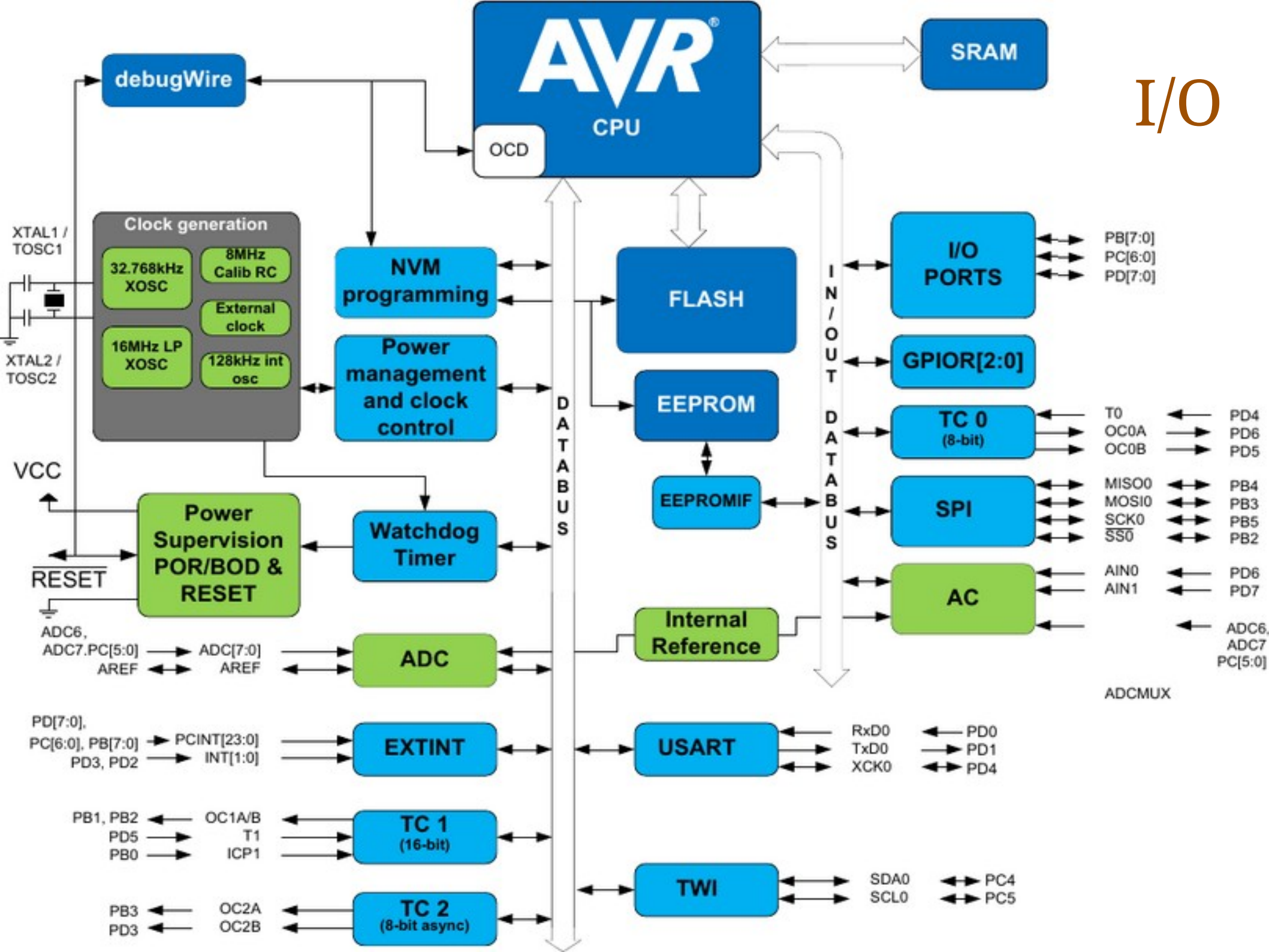
A maioria das instruções podem acessar qualquer registrador e completar uma operação em um ciclo de clock da CPU

Existe também o Stack Pointer de 16 bits

Os últimos 3 pares de registradores podem ser usados como ponteiros de endereço de 16 bits para endereçamento indireto do espaço de dados

X-register Low Byte	
X-register High Byte	
Y-register Low Byte	
Y-register High Byte	
Z-register Low Byte	
Z-register High Byte	

I/O


AVR
CPU
OCD
SRAM
debugWire
Clock generation
32.768kHz XOSC
8MHz Calib RC
External clock
16MHz LP XOSC
128kHz int osc
XTAL1 / TOSC1
XTAL2 / TOSC2
NVM programming
Power management and clock control
FLASH
EEPROM
EEPROMIF
DATABUS
IN/OUT DATABUS
I/O PORTS
PB[7:0]
PC[6:0]
PD[7:0]
GPIOR[2:0]
TC 0 (8-bit)
T0
OC0A
OC0B
PD4
PD6
PD5
SPI
MISO0
MOSI0
SCK0
SS0
PB4
PB3
PB5
PB2
AC
AIN0
AIN1
PD6
PD7
ADC6, ADC7 PC[5:0]
ADCMUX
VCC
Power Supervision POR/BOD & RESET
RESET
Watchdog Timer
Internal Reference
ADC6, ADC7.PC[5:0]
AREF
ADC[7:0]
AREF
ADC
PD[7:0], PC[6:0], PB[7:0]
PD3, PD2
PCINT[23:0]
INT[1:0]
EXTINT
USART
RxD0
TxD0
XCK0
PD0
PD1
PD4
PB1, PB2
PD5
PB0
OC1A/B
T1
ICP1
TC 1 (16-bit)
TWI
SDA0
SCL0
PC4
PC5
PB3
PD3
OC2A
OC2B
TC 2 (8-bit async)

Watchdog

Ajuda evitar alguns problemas como travamento

Possui oscilador independente

Quando habilitado reinicia o sistema a cada 16 ms (típico)

- A ideia é que o temporizador seja zerado em intervalos de tempo para evitar o reinício
- Quando tem algum problema o contador não é zerado
- Não evita todos os problemas

Portas I/O

Interface do controlador com o mundo externo

Adquire dados

Aciona processos

Recebe ou gera níveis lógicos de tensão

Ocupam a maioria dos pinos do microcontrolador

Cada elemento pode ser configurado por software

Portas I/O

Saída

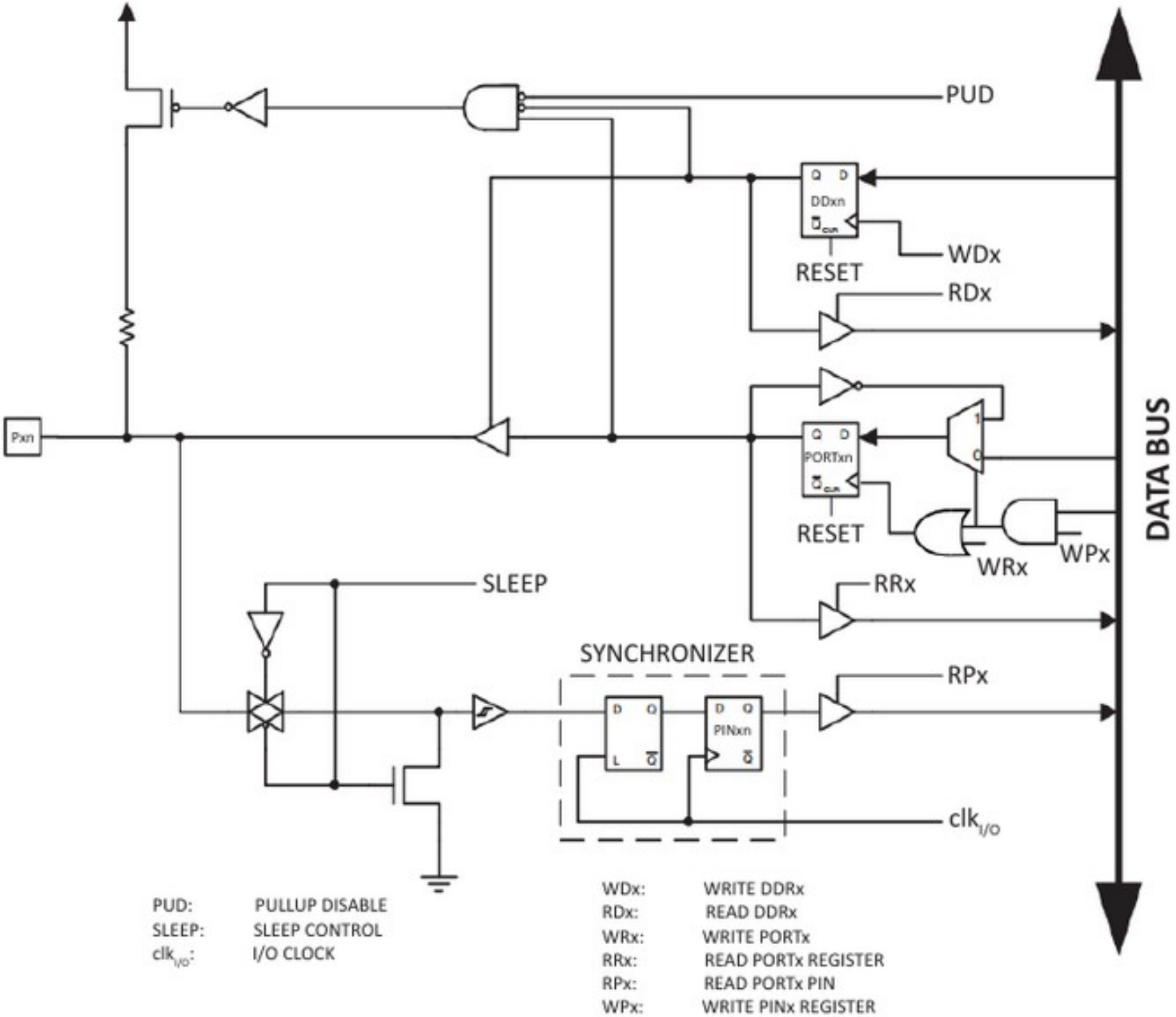
- Transmitir um dado que esteja presente no barramento de dados ao ambiente externo
- São utilizados latches que retêm os dados

Entrada

- Recepção de um dado externo para o barramento
- Usa buffers

Para armazenar as informações são usados registradores

I/O


PUD
DDxn
RESET
WDx
RDx
Pxn
PORTxn
RESET
WRx
WPx
DATA BUS
SLEEP
RRx
SYNCHRONIZER
RPx
PINxn
clk I/O
PUD: PULLUP DISABLE
SLEEP: SLEEP CONTROL
clk I/O: I/O CLOCK
WDx: WRITE DDRx
RDx: READ DDRx
WRx: WRITE PORTx
RRx: READ PORTx REGISTER
RPx: READ PORTx PIN
WPx: WRITE PINx REGISTER

ATmega328

Possui registradores de 8 bits

Portas divididas em 3 conjuntos (Porta B, Porta C e Porta D)

Até 8 dispositivos I/O (PB0, PB1, ..., PB6 e PB7)

Cada bit do registrador determina a função de um pino

Bit no nível 1 configura para entrada, 0 é saída

Os pinos são <u>multiplexados</u> com timers, USART, etc.

Especificação I/O

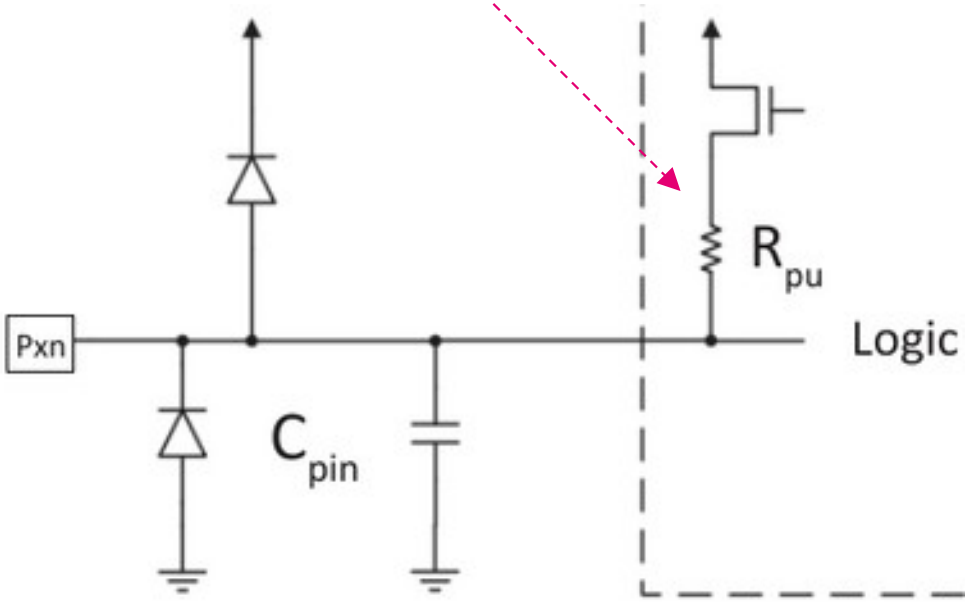
Corrente máxima em cada pino: 40 mA

A soma das correntes de todo CI: 200 mA

Os pinos podem ter resistências pull-up para 5V

Útil para botões, interruptores, etc.

Possui proteção contra sobre/subtensão



Memória ATmega328

RAM 2 kB (8-bit)

- Volátil

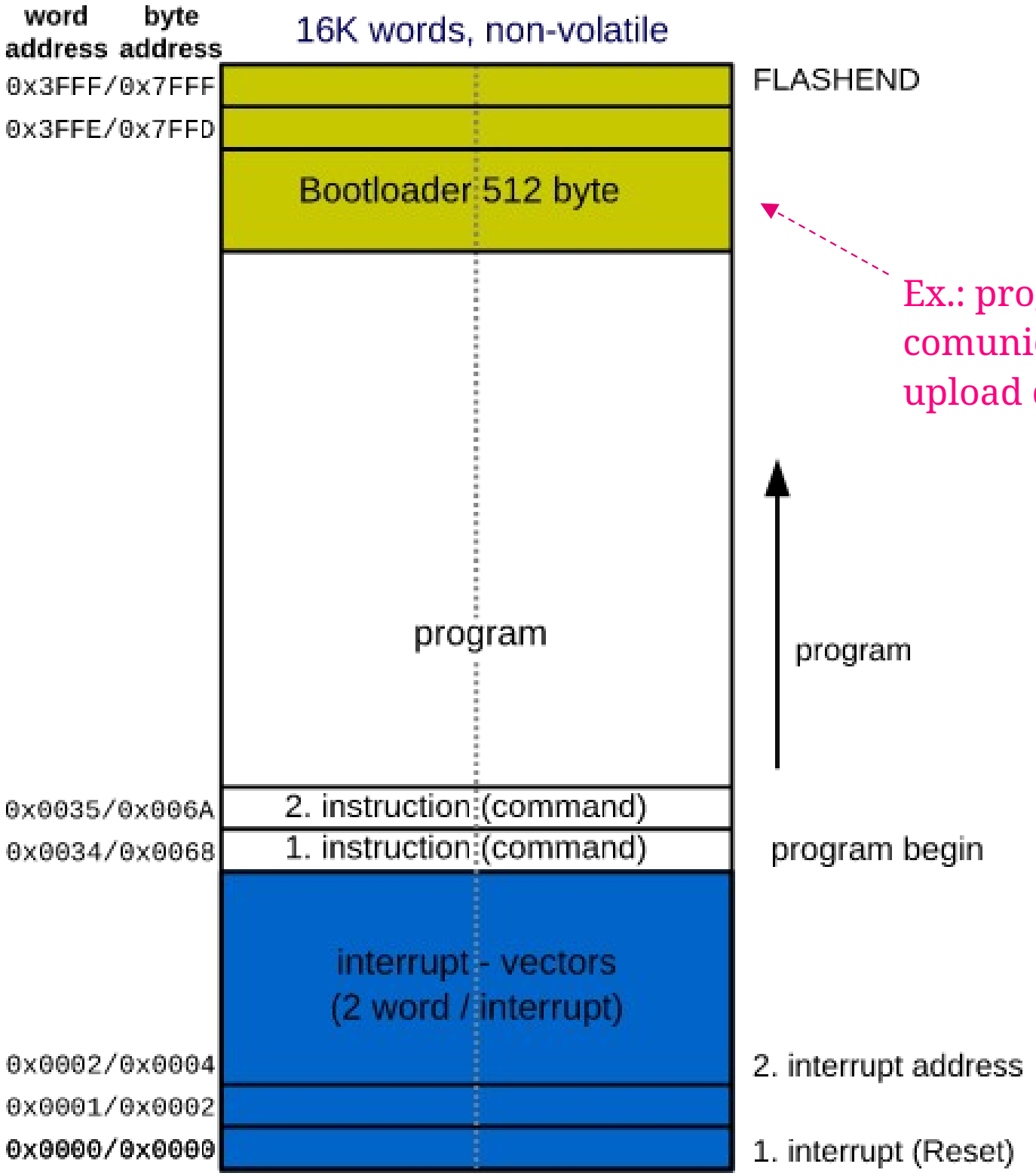
EEPROM 1 kB (8-bit)

- Não volátil, ciclos de escrita/leitura: 100.000
- Cada byte pode ser escrito/lido individualmente a cada vez
- Espaço separado de armazenamento

FLASH 32 kB (16 kB words) ← instruções têm 16 ou 32 bits

- Não volátil, ciclos de escrita/leitura: 10.000
- Destinado a poucos ciclos, ex.: código do programa

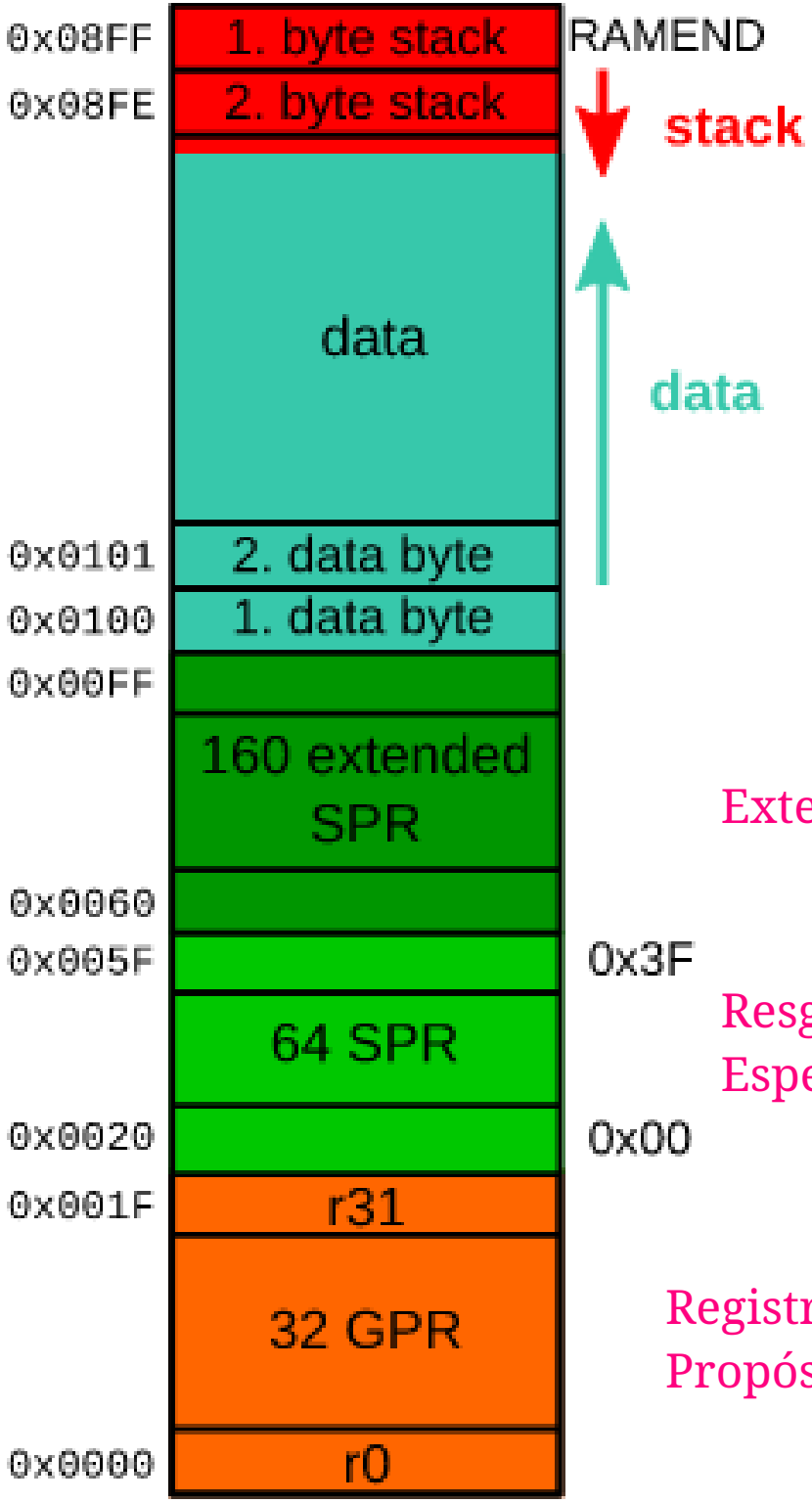
Flash



Ex.: programa especial que faz a comunicação com o PC e permite upload de novos programas

https://www.weigu.lu/tutorials/microcontroller/04_memory/index.html

https://www.weigu.lu/tutorials/microcontroller/04_memory/index.html



SRAM (2.25KiB)

2303 byte, volatile

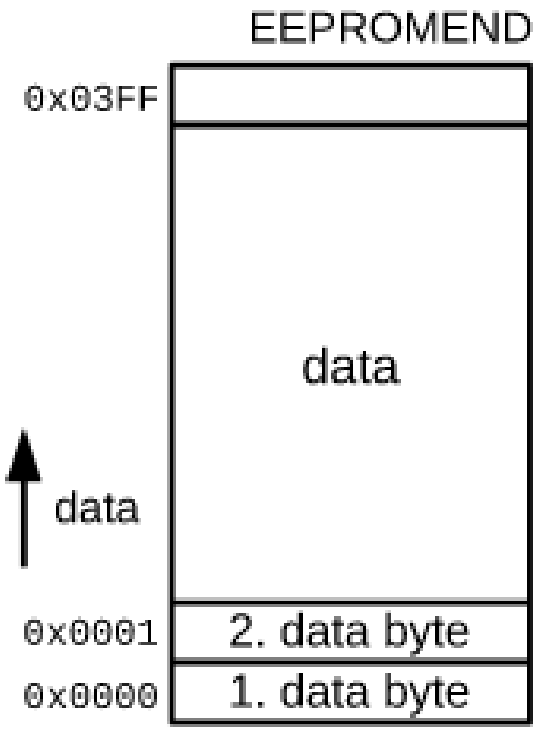
Extensão do SPR

Resgistradores de Propósito Especial (SPR). I/O, etc.

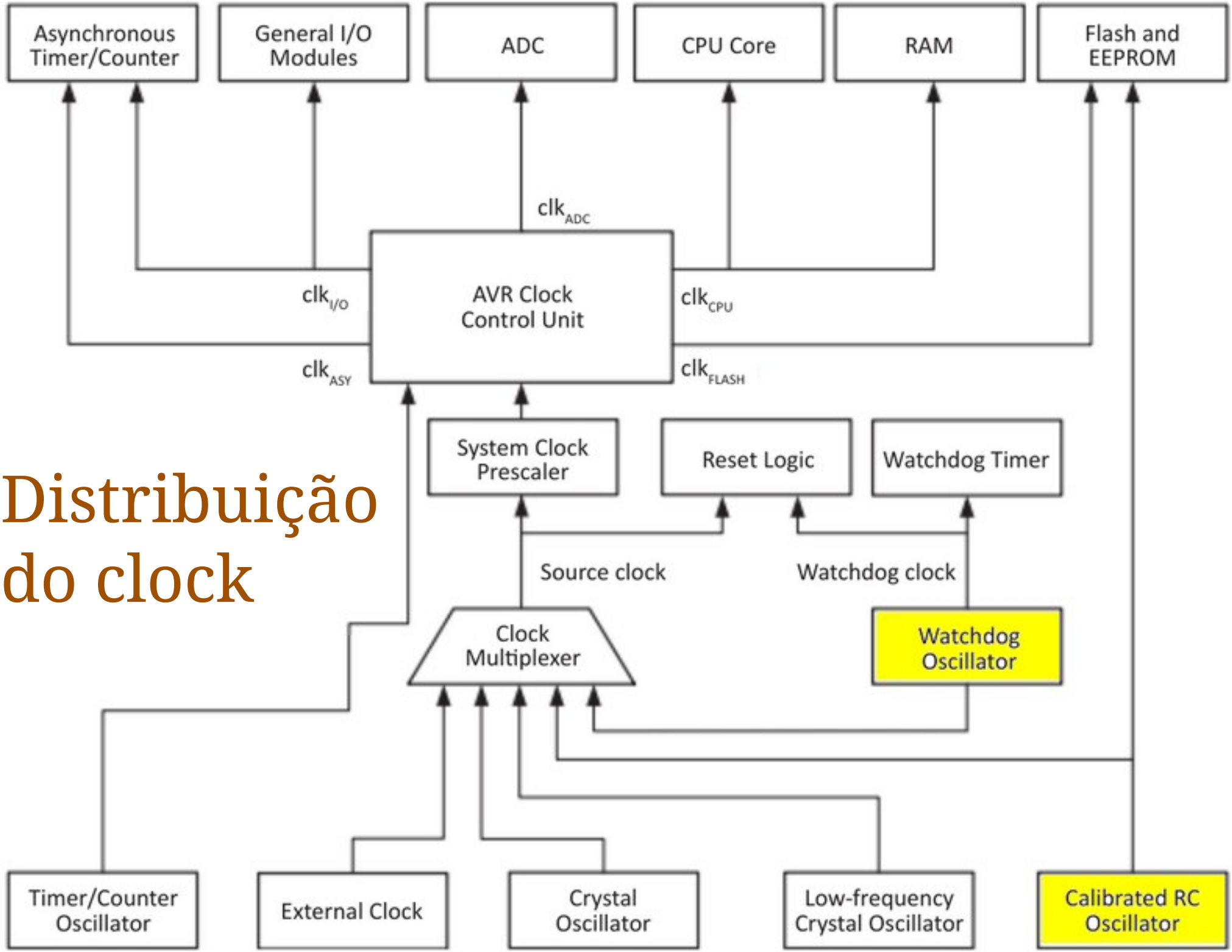
Registradores de Propósito Geral (GPR)

EEPROM (1KiB)

1024 byte, non-volatile



Distribuição do clock



O μC executa as instruções por meio de uma base de tempo intitulada ciclos de máquina

Cada instrução possui um tempo de execução que é um múltiplo de um ciclo de máquina

Esse sinal oscilante é chamado de sinal de relógio (clock) e é proveniente de um circuito oscilador

É possível usar o oscilador interno do μC, ou um externo

Arduino UNO (ATmega328)

Usa por padrão oscilador externo a 16 MHz

O clock máximo do ATmega328 é 20 MHz

O ATmega328 possui internamente

- Oscilador RC calibrado de 8 MHz (ou 1 MHz, programável)
- Oscilador de 128 kHz (usado pelo Watchdog)

Clock externo

- Mais comum: oscilador de cristal, que possui boa precisão

Prescaler

- Usado para diminuir a frequência do sinal de clock primário

Arquitetura RISC

Processador

Conjunto de instruções (*instruction set*)

Instruções compreensíveis para um um processador

O programa que o processador executa é gravado nos bytes da memória no formato de código de máquina

asm	bin	hex
ldi r16, 0x20	1110 0010 0000 0000	e200
out 0x04, r16	1011 1001 0000 0100	b904
out 0x03, r16	1011 1001 0000 0011	b903
ldi r17, 0xff	1110 1111 0001 1111	ef1f
dec r17	1001 0101 0001 1010	951a
brne -2	1111 0011 1111 0001	f7f1
rjmp -5	1100 1111 1111 1011	cffb

Assembly

Linguagem de baixo nível que tem correspondência direta com as instruções do código de máquina da arquitetura

Com uma tabela de conversão simples é possível manualmente obter o código de máquina

Não é necessário um compilador

O Montador Assembler é um programa que converte Assembly em código de máquina

O montador e a linguagem possuem adicionamente vários recursos para facilitar a vida do programador

O programa gravado na memória do μC especifica as instruções que devem ser executadas em ordem

Existem vários tipos de instruções

- Carregar dado da memória em um registrador
- Realizar operação de soma de dois registradores
- Etc.

A definição da quantidade e da função de cada instrução depende da arquitetura que o sistema digital possui

RISC E CISC

Geralmente os conjuntos de instrução são divididos em duas grandes categorias

- RISC (Reduced Instruction Set Computer)
- CISC (Complex Instruction Set Computer)

RISC

- Conjunto pequeno de instruções simples com largura fixa
- Otimização na execução de chamadas de funções
- Pequena quantidade de modos de endereçamento
- Grande uso de *pipelining*
- Uma instrução por ciclo

As premissas do desenvolvimento do RISC levaram em consideração um conjunto reduzido e simples de instruções

- Podem eventualmente ocupar mais RAM
  - Necessidade de mais instruções do que uma arquitetura CISC

Arquitetura simples, mais fácil de fabricar, mais barato

Menor tamanho físico do chip, menor dissipação de calor

Instruções de tamanho fixo

- Simplicidade de busca da instrução pelo processador

A arquitetura CISC VAX possui cerca de 300 instruções

A arquitetura RISC SPARC possui cerca de 64 instruções

- Largura fixa de 4 bytes

Chamadas de sub-rotinas usam registradores para armazenar dados e endereços

RISC usa poucos modos de endereçamento para acesso a memória

- Geralmente 2 tipos de instruções, load e store

RISC usa uma maior quantidade de registradores

RISC usa bastante *pipelining*

Instr. No.	Pipeline Stage						
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX
Clock Cycle	1	2	3	4	5	6	7

wiki.edunitas.com

IF = Instruction Fetch, ID = Instruction Decode, EX = Execute, MEM = Memory access, WB = Register write back

RISC

Uma instrução por ciclo (ou pelo menos a maioria delas)

- Uma das características mais importante

Internamente o processador não utiliza microprograma

- Traduz instruções em sequências de operações
- Permite dividir instruções complexas em simples

MIPS

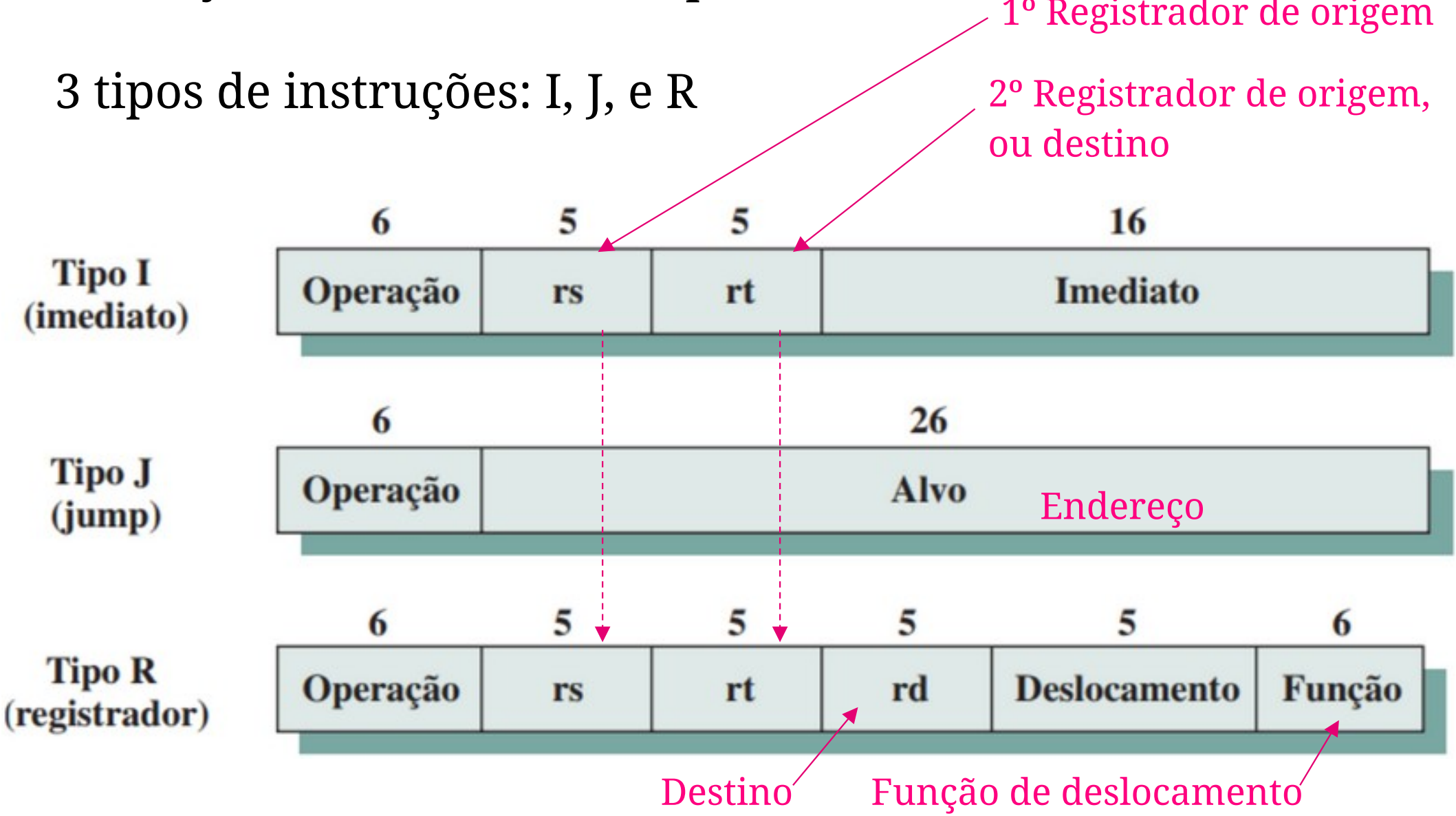
- Um dos primeiros processadores
- Lógica de execução simplificada
- Dividido em 2 partes: CPU e gerenciamento de memória
- Todas as instruções usam palavra de 32 bits
- Apenas LOAD/STORE para operações com memória

MIPS

Codificação das instruções (32 bits)

Instruções condicionais simplificadas

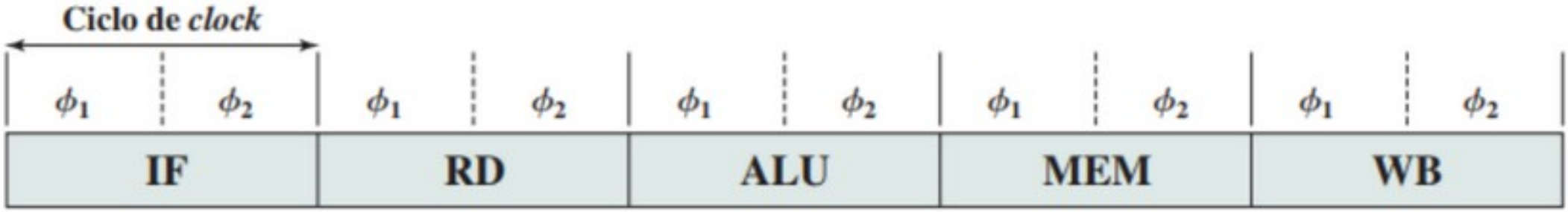
3 tipos de instruções: I, J, e R



MIPS

5 estágios de *pipeline*

- IF – busca por instrução
- RD – busca por operandos
- ULA - operação ULA, ou geração de endereços
- MEM – referência a dados de memória
- WB – atualização do banco de registradores



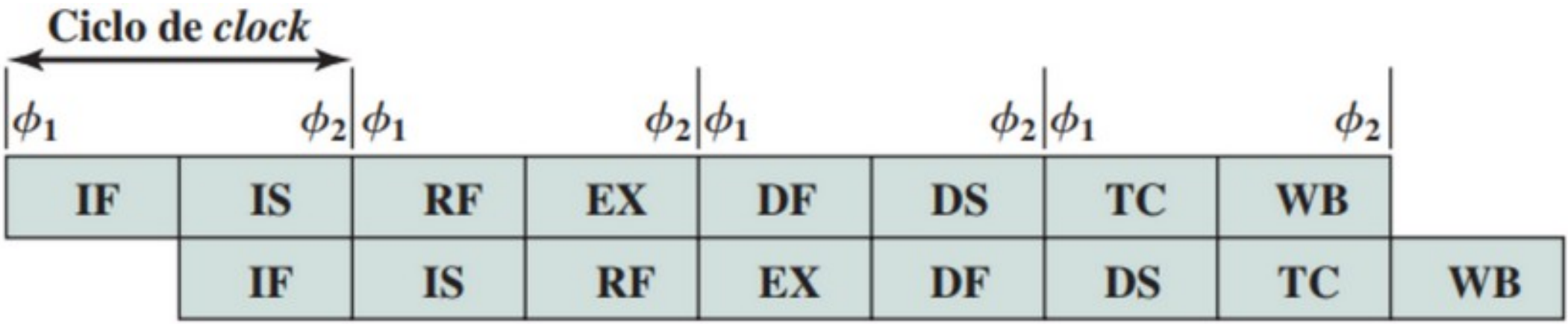
MIPS

Algumas operações podem ser executadas em meio clock

- Aproveitando alguns recursos do pipelining

R4000 é o modelo de processador de maior capacidade

- 64 bits, mais rápido, maior cache, pipeline de 8 estágios



(continua no próximo slide)

Quem usa

Roteadores

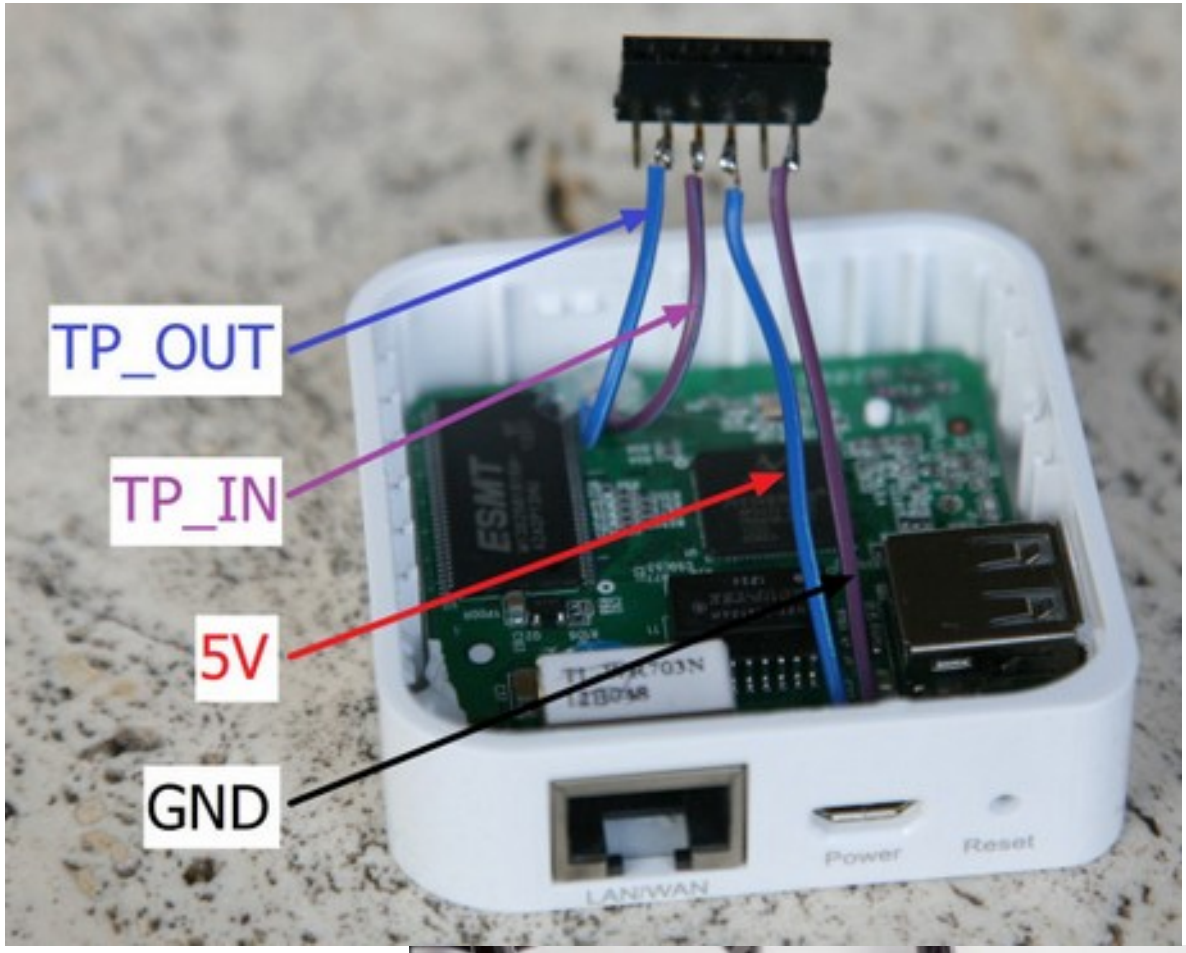
- Podem usar a distribuição Linux OpenWRT

PlayStation, Nintendo 64

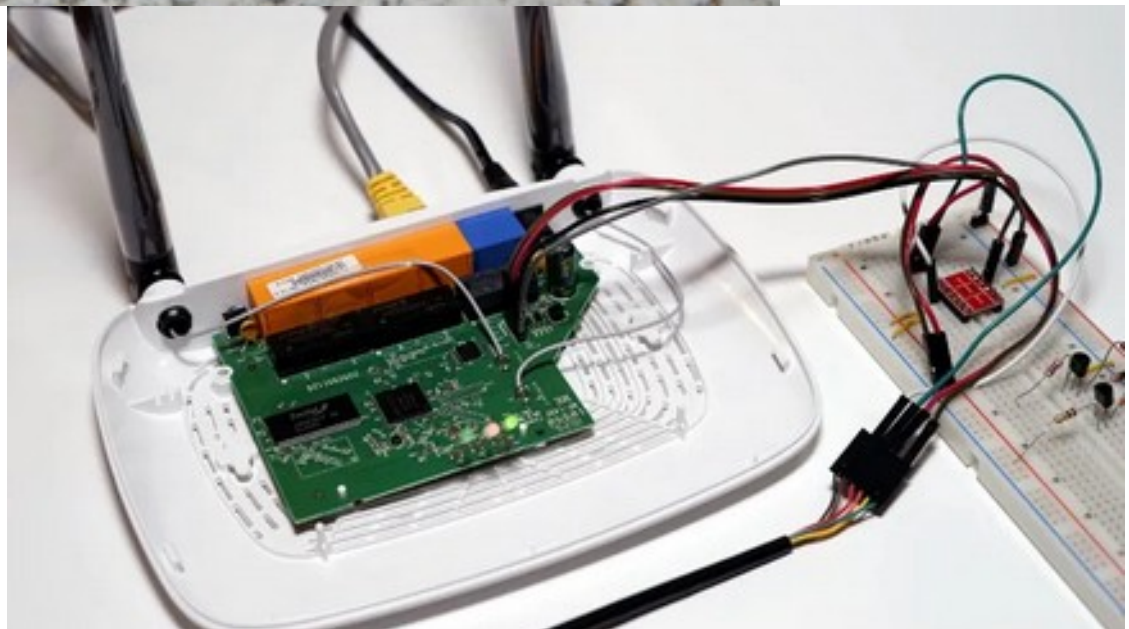
Supercomputadores chineses

- Processadores Loongson

Satélite Beidou


TP_OUT
TP_IN
5V
GND
ESMT
LAN/WAN
Power
Reset

tp-link



Exemplo de aplicação

Mozilla Firefox

File Edit View History Bookmarks Tools Help

http://192.168.0.5/cgi-bin/temperature

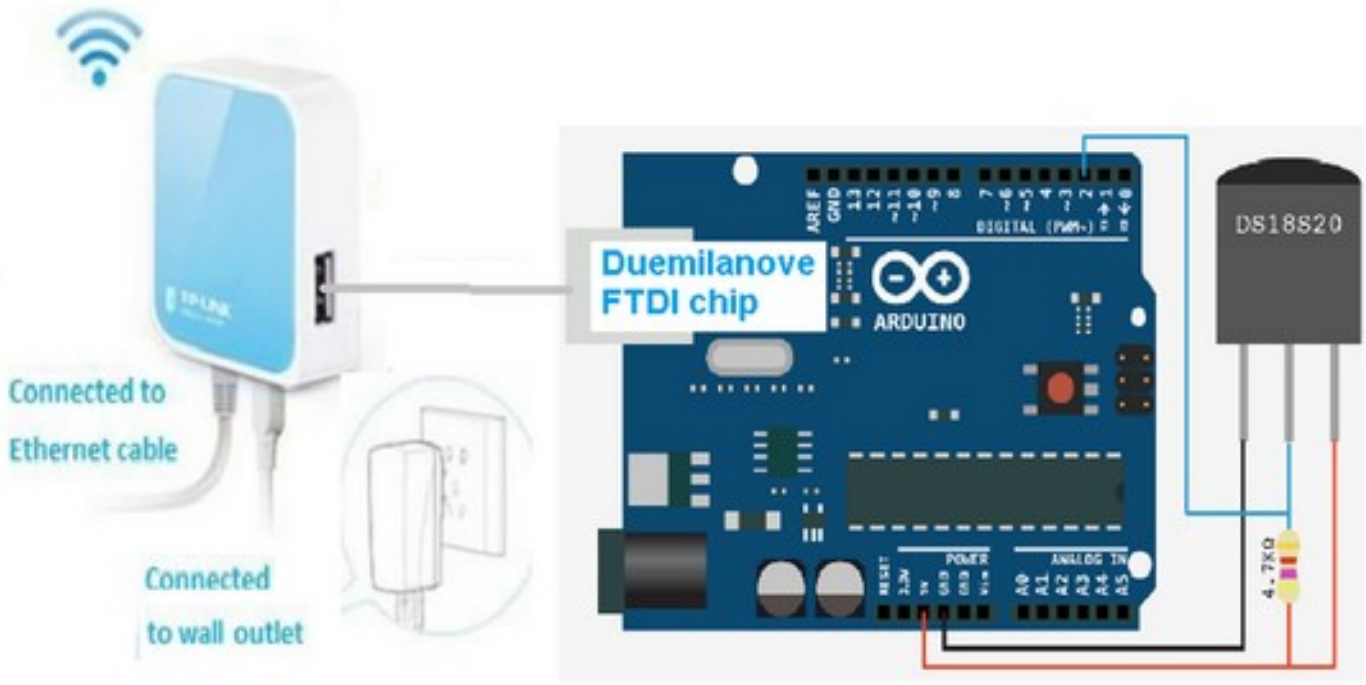
ds18b20 arduino

http://192.168....bin/temperature | OpenWrt - LuCI

WiFi thermometers and signal inputs

realized by pocket router TP-Link WR703N + Arduino

```
Read pin 3 digital in -> LED pin 13
0
Read pin 4 digital in
0
Analog input 0
375
Analog input 1
325
Dallas Temperature IC Control Library Demo
Locating devices...Found 3 devices.
Parasite power is: ON
Device 0 Address: 284834BB030000B0
Device 1 Address: 28CC5F0E0200005C
Device 2 Address: 284E6761030000C3
Requesting temperatures...DONE
Device Address: 284834BB030000B0 Temp C:
25.19
Device Address: 28CC5F0E0200005C Temp C:
25.25
Device Address: 284E6761030000C3 Temp C:
25.19
```



```
$ ssh root@192.168.84.248
BusyBox v1.35.0 (2023-01-03 00:24:21 UTC) built-in shell (ash)

  _______                     ________          __
 |       |.-----.-----.-----.|  |  |  |.----.|  |_
 |   -   ||  _  |  -__|     ||  |  |  ||   _||   _|
 |_______||   __|_____|__|__||________||__|  |____|
          |__| W I R E L E S S   F R E E D O M
 -----------------------------------------------------
  OpenWrt 22.03.3, r20028-43d71ad93e
 -----------------------------------------------------
root@OpenWrt:~# cat /proc/cpuinfo
system type             : MediaTek MT7628AN ver:1 eco:2
machine                 : TP-Link TL-WR840N v4
processor               : 0
cpu model               : MIPS 24KEc V5.5
BogoMIPS                : 385.84
wait instruction        : yes
microsecond timers      : yes
tlb_entries             : 32
extra interrupt vector  : yes
hardware watchpoint     : yes, count: 4, address/irw mask:
[0x0ffc, 0x0ffc, 0x0ffb, 0x0ffb]
isa                     : mips1 mips2 mips32r1 mips32r2
ASEs implemented        : mips16 dsp
Options implemented     : tlb 4kex 4k_cache prefetch mcheck ejtag
llsc pindexed_dcache userlocal vint perf_cntr_intr_bit perf
shadow register sets    : 1
```

Exemplo de distribuição Linux para a arquitetura MIPS rodando em um roteador